

Virtual Reality Operating Room

Type of Project: Engineering
Level: Graduate

Nicholas Weber

Ali PiyarAli

Josef Svoboda

Summary

Virtual reality (VR) has potential applications in the field of medical education and healthcare. Our project involves simulation of a hospital operating room where students can familiarize themselves with their surroundings and interact with objects by picking them up and dropping them. The experience's core task involves picking up objects fallen on the room's floor and bringing them to a collection point at the center of the room. We hope the VR experience will enhance students' knowledge and comfort level in complex healthcare settings.

Project Repository and Demo

- **GitHub Repository:** <https://github.com/noweber/DGMD-E-14-Final-Project>
- **Demo:** <https://www.youtube.com/watch?v=YrA0gC6HJTM>

1 Introduction

Virtual reality (VR) is an emerging and fascinating field with exciting applications in the field of medical education and healthcare. VR allows for the user to interact within a 3D computer generated immersive environment. Usually a specific hardware such as a reality headset is required for the user to perceive his or her surroundings. In medicine, students require simulation and exposure to hospital settings, clinics and operating rooms so that they can develop familiarity and comfort with patient encounters. In addition to real life exposures, VR can provide an excellent source of medical education training.

The goal of our project is to immerse the students in a hospital operating room where the users can familiarize themselves with their surroundings and complete tasks such as putting together a surgical tray. We hope to first create a virtual training platform on the computer followed

by integration with VR hardwares such as Meta Quest 2 or Google Daydream. Medical and nursing students can use these devices to increase their exposure to operating rooms prior to or alongside their in-person experiences. We hope the VR experience will enhance students' knowledge and comfort level in complex healthcare settings. We see great potential in VR for medical education and hope to demonstrate how VR can provide a stimulating and helpful platform for students.

2 Background

Virtual reality (VR) is a digital experience where people use wearable technology to interact with the application in a way that seems real. There are several different manufacturers of this hardware and some employ different techniques to deliver these experiences to users. Eye tracking, the types of controllers, and the types of displays are just a few of the differences. The commonality is the progression towards providing a simulated experience in a digital setting to the user indistinguishable from its counterpart in the physical world.

Many of the technologies used in developing virtual reality applications are derived or shared with modern video game technologies. However, VR has many applications beyond entertainment. Often called "serious" games, the application of game designs and technologies for other purposes is starting to be seen in many industries. One such example is the development of surgeon simulation games within healthcare industries where trainees use VR applications to hone their skills. Serious games have the potential to create a very entertaining way for people to build their work skills to benefit their customers or patients. Virtual reality offers the chance for this even in the absence of expensive training facilities.

Using a real-time 3D editor such as Unity (commonly referred to as a game engine), designers, engineers, and

artists can work together in an environment which integrates their workflows. Unity allows the designers to manipulate the game world and run the application on their personal computer, integrated with the latest code and art assets, without having to wait for a build. The engineers can write code within their IDE of choice and this is automatically integrated back into the Unity project. The artists can edit assets in a 3D modeling tool, such as Maya, or create 2D assets, perhaps in Photoshop, which are also integrated into the project and populated with the variables created by the engineers. The Unity libraries handle building and porting between different devices such as mobile or VR. The end result is that applications created in a platform such as Unity integrate workflows from different specializations and reduce the burden of specialized portability engineering.

Our team's process is to design and engineer the application using the Unity platform while importing art assets from the Unity Asset Store. We will host the code repository on GitHub with local development occurring in the Unity editor and Visual Studio. Each developer can build locally and we can automate builds for different platforms via GitHub Actions when code is committed. We will not continuously deploy this application to any app store, but instead deploy locally via USB. Features will be developed and integrated (more details within the Methodology section below) as a 3D application. Using Unity to provide access to the VR hardware APIs, the 3D application will be integrated to be experienced in VR.

3 Literature Review

1. Preliminary Study of VR and AR Applications in Medical and Healthcare Education: This study shows various applications and devices of virtual reality (VR), augmented reality (AR), and mixed reality (MR) in healthcare and medical education. Examples include VR exposure therapy for phobias and AR autism treatment to help interpret human emotions. In medical education, Taiwan's Tainan University of Technology created the first virtual anatomy platform for anatomy teaching and the Taipei Veterans General Hospital has developed VR surgery simulation training. The challenges of using VR, AR and MR in health care include costs, integration with medical databases, and medical staff training.

2. Evaluation of a Slipped Capital Femoral Epiphysis Virtual Reality Surgical Simulation for the Orthopaedic Trainee: Twenty-one orthopedic trainees were randomized between three control groups. These groups were for traditional training forms and one group used a VR training simulator. Each form of training produced similar results and there was no observed difference in mean surgical times or other

metrics. However, the VR training was rated higher by participants, suggesting VR could be a useful adjunct to surgical training.

3. Easy Unity VR Tutorial for Beginners: The tutorial shows how to create a simple VR scene in unity with XR (AR/VR/MR). It shows how to configure VR inputs including input actions such as grabbing objects, moving right or left, and locomotion system (teleportation area and continuous move). One of the benefits of this tutorial is that it requires no coding, it uses built in Unity settings.

4. Get Started with Oculus in Unity: This developer documentation from Meta shows the process for creating and integrating a simple virtual reality game in Unity with the Meta hardware. The concepts of 3D environments, cameras, physics, and other project elements are shown which could be extended to create an operating room. With a scene created, it covers the basics of building and running the app on the Oculus wearable hardware after integration with the SDK in Unity. We can follow this general path while extending it to meet our specific goals.

5. A Guideline Study for Designing Virtual Reality Games: This article describes how game technologies can be adapted to many different use cases. Although, in doing so, sometimes issues arise such as motion sickness. Additionally, the user or player's perception tends to be different than physical games or even digital games which use different mediums. This guideline article discusses design pillars we can reference to create an engaging immersive experience and attempt to minimize issues regarding controls, perception, and navigation. Case studies map the game elements which we can compare to our own in learning from.

4 Methodology

This project created a 3D virtual reality (VR) application using the Unity real-time 3D development platform and tools (commonly referred to as a game engine). The game runs as a standalone PC game, but also integrates with VR hardware (the Meta Quest 2).

The preparation of the operating room is a highly procedural process, each specific surgery requires a different setup and training for correct organization. The use of real operating rooms is expensive, and in many teaching establishments not all operating procedures are feasible, therefore a simulation game would extend accessibility beyond current teaching hospitals.

In order to achieve this, the player can pick up objects and move them to the surgical tray area which simulates the pos-

sible process of preparing the operating room (OR) level for an upcoming surgery. Each operating room is procedurally-generated – the placement of each object will be different each time.

4.1 Requirements

The project is developed in Unity, a cross-platform game engine, which integrates features such as customizable physics and graphics, importable assets such as third-party game objects, integration with VR/AR and others. Unity implements an entity-component architecture and this project's component scripts are written in C#. At a high level, the application development requires defining game objects, configuration of a 3D scene, gameplay scripting, and integration of the desktop application with a VR headset (Meta Quest 2).

An operating room scene requires defining game objects for placement within world space. Re-usable game object definitions are known as prefabs within Unity and can be instantiated multiple times. Both player movement and object manipulation require understanding of the physics libraries embedded within Unity. In the centre of the operating room scene, the operating table is placed. Objects are then placed within the room and the player can move freely within the scene's constraints.

The project is created in Unity version 2021.3.13f1. Installation instruction can be accessed via the following link: [Install the Unity Hub and Editor](#).

4.2 Operating Room Scene

The Unity project comes with an empty grid scene with a main camera and directional light object. A camera is an object through which the user views the virtual world. It can be adjusted to change the user point of view and comes with several built-in properties and methods. The Unity game engine approximates the real-world lighting behaviour. Default directional light object behaves like a sunlight and does not have any identifiable source and therefore can be placed anywhere in the scene.

The initial step in creating a 3D Operating Room (OR) scene requires adding elements to the Unity empty grid scene. By importing and using Unity standard assets, the following elements were added to the scene: floors, walls, and first-person player movement. The next step in setting up an OR scene was to import OR room assets from the Unity Asset Store. Below are the list of some of the assets that were imported:

- Devices
 - Anesthesia monitor
 - Vital Sign monitor (blood pressure, heart rate, ECG, EKG, etc.)
 - Electronic Manifolds (medical gas monitoring for life support such as oxygen)
 - Foot pedals
 - Lamp arm
 - Microscope and microscope screen
 - Monitor, mouse, and keyboards
 - Telephone
- Furniture
 - Hospital Bed
 - OR Tray
 - Steel table
 - Steel movable tray
 - Stool
 - Suture Box
 - Table / Desk
 - Trolley
 - Movable shelf and storage systems
- Props
 - Alarm bell
 - Arm pillows
 - Desk litter (documents, stapler, stationeries)
 - Door sensor
 - Intercom
 - Syringe Pack and holder
 - Screen wall mount bracket
 - Medical waste bin
 - Inputs such as electrical, cable, etc.
 - Fire Blanket
 - Radio
 - Coffee Mug
 - Surgical Instruments (Forceps, Scissors, Retractors, Travers, Scalpel, etc.)
- Texture (colors, pattern, design for above assets including tiling for floors)

In Unity, when an object is to be visible in 3D space it requires a MeshFilter and MeshRenderer component. This gives the impression of a solid object when viewed by the camera and describes the shape. Materials can be applied to the mesh such that the appearance of a surface in the scene can be metallic or rocky. A custom mesh can be

made in third-party software and imported to create a new shape or asset. All imported assets listed above have meshes. The mesh does not necessarily determine the shape of the collider, however, which allows an object to collide with another object. For example, a 3D bounding cube or sphere can be used around a mesh to provide collision.

The next step was to create a Prefab of *GameObjects* needed to place and spawn in the operating room. In Unity, a *GameObject* is any object in the Unity scene that can represent characters, props, scene, camera, assets, etc. Prefab is a process of defining Unity *GameObject* as a template or reusable assets. In object-oriented programming language this is similar to instantiating a class as an object. Each entity may have many components each of which may have many classes involved. In this way, a *GameObject* can be used in multiple places with the same settings while allowing each to have its own state. Settings can be overridden on an individual prefab instance, which is called a variant, a concept similar to inheritance in object-oriented programming. Prefabs also allowed us to apply the same scripts to all the prefab objects. For this project, any asset that is considered "placeable" was added to the objects which populate the scene as a component script. These would be devices, furniture, and props such as coffee mug, surgical instruments, etc.

Next step was to add physics to the prefabs. In Unity this is achieved by configuring items in the scene as rigid bodies and then manipulating the rigid body settings such as mass, drag, gravity, collision detection type, etc. This allows the Unity physics engine to simulate physics forces.

4.3 Scripting Movement

(Unity FPS Controller, 2019)

The movement of the player is scripted in the style of a First Person Shooter or first-person point of view.

The keyboard and mouse input determine the movement are:

- W, A, S, D to walk,
- Mouse to look around,
- Space to jump, and
- Left Shift to sprint.

The steps to create a first-person controller are:

- Create a first-person player game object.
- Move the main camera inside the first-person player object.

- Create a script for first-person player game object.
- Define the walk speed, running speed, jump speed, fall speed, and lock speed variables.
- Define the player camera variable.
- Define the start function - get the initial player position.
- Define the update function - depending on the input controls, modify the Unity object transform function. *(Every unity object has transform function that is used to store and change the objects position and rotation.)*
- Attach the script to the first-person player game object item Assign the main camera to the player camera variable under the unity settings (inspector window)

4.4 Object Placing

In the centre of the OR of the scene is an operating table with specified machines. To create gameplay, the centre of the OR will always be the same, but all the other objects will be placed randomly using Poisson Disc Sampling. An array will be created for objects that are pickup-able to create a surgical tray or bring them to some collection point. Another array will be created for objects that are not pickup-able such as furniture. Objects in the scene are in three dimensions: x, y, z. Dimensions x and z are two dimensions representing length and width and dimension y is 3D representing height. For all the objects, y-dimension is provided. For non-pickup-able objects, the height of the floor dimension is zero. All the pickup-able objects for this game will be on a table and the height of the table is used as y-dimension.

To place the object randomly, the Poisson Disc Sampling ([Fast Poisson Disk Sampling in Arbitrary Dimension](#), 2007) function is written and applied to pickup-able and non-pickup-able prefab game objects. The function takes an array of objects and the bounding x, y dimensions of the scene where the objects require placing. The function generates x, y coordinates within the bounding x, y dimensions and then checks if an object is already present. If yes, then it generates another coordinate, otherwise it places the object and iterates through the array to the next object.

4.5 Manipulation of objects

The manipulation of objects has two parts - scripting the ability to identify and move an object - pick up, carry, drop off - and creating a collection point for the object to be carried to. This is achieved by spawning a sphere - a particle effect - which when it detects an object being carried to it makes the object disappear, generating an effect such as a sound.

The C# script for manipulation of objects is a pickup controller – it enables the player to pick up and drop objects: when the player presses the "Fire1" button, the script shoots a ray in front of the player and attempts to pick up any object within a certain range and layer mask. If the object has a Rigidbody component, it is picked up and its gravity is disabled. If the player is already holding an object, pressing "Fire1" again will drop the object. The script also moves the object towards the player's camera when it is picked up and enables and disables the object's gravity and constraints as necessary. The script has a timer that prevents the player from rapidly picking up and dropping objects. The layer mask and pickup range can be adjusted to specify which objects can be picked up and how far away they can be picked up from.

The script for disappearing objects is a pickup collector: it's attached to the GameObject in the Unity scene, when the game object collides with another object with a Rigidbody component, the script checks if the SuccessParticles prefab is assigned. If it is, the script creates an instance of the prefab and assigns it to the transform of the game object. The script then destroys the object that was collided with.

In the OnCollisionEnter method, the script checks if the collided object has a Rigidbody component by calling GetComponent on the game object with the type parameter set to Rigidbody. If the returned value is not null, the script knows that the object has a Rigidbody component, so it proceeds to create an instance of the SuccessParticles prefab and assigns it to the game object's transform. It then destroys the collided object.

Here is an example of how the script could be used:

- The game object with this script attached could be a surgical tray in a game level, and the objects that the player must collect drop onto the tray such as scalpels could have a Rigidbody component attached.
- When the player successfully drops an object onto the tray, the SuccessParticles prefab will be created and the object will be destroyed. This could be used to give feedback to the player that they have successfully completed the objective.

4.6 Hardware Integration

(Get Started with Oculus in Unity, 2022)

The desktop application is integrated with Meta Quest 2. Hardware requirement for Meta Quest 2 are 2.0+ GHz processor and 2 GB system RAM. The software system requirements are Windows 10 (64-bit only) or macOS Sierra 10.10 or higher (x86 only – limited features), and minimum

Unity version of 2020.3 LTS.

Headset setup:

- Download the Oculus mobile app from Google Play or the App Store.
- Follow in-app instruction and create/sign-in with your developer account credentials.
- Pair the headset with your phone.
- Wear the headset. Follow the instructions that appear in the headset to complete the setup

Software set up:

- Install Android SDK (minimum: SDK 19 – Android 4.4 or KitKat and SDK 25 – Android 7.1.1 Nougat)
- In the Unity Hub, go to Installs and add Android SDK & NDK Tools and OpenJDK modules.
- Using the Oculus mobile app settings, set the headset into developer mode.
- Install the Meta Quest Developer Hub companion tool to the PC.
- Connect the headset to the computer using a USB-C cable and put on the headset and click allow.
- Verify the connection by going to Unity build settings and under Android platform Run Device setting, ensure that Oculus Quest is selected.
- Click on Player Settings and select XR Settings and click the checkbox the *Virtual Reality Supported*. Then under the Oculus, check the *V2 Signing (Quest)*. Then close the player and build settings.

Integration:

- From the Unity Asset store, download the Oculus Integration package.
- In the Asset folder, go to the Oculus Asset Package, then VR, then Prefabs and select the OVRPlayerController and drag it to the existing scene where the player will start.
- Open the OVRPlayerController and select the OVRCameraRig. In the Inspector window, expand OVR Manager (script) and set the target device as Quest.
- Final step is to Build and Run (in Unity go to File, then Build Settings and then Build and Run). You should see the app running on the headset.

After the integration is complete, the app can be run using the following steps:

1. Close Unity, then start the Oculus app on your computer to ensure the link between Unity and the hardware occurs.
2. Open Unity and load the project ensuring that the 2021.3.13f1 version is selected.
3. Reconnect the VR headset to the development computer via USB after opening the Oculus app and Unity.
4. Headset will prompt for permissions (to update files and enable Oculus Link, for example). Allow the permission for the app to deploy via USB.
5. Load the scene "ORScene" and press the play button

4.7 Deliverables and Timeline

1. Oct 16, 2022
 - (a) Unity Project & Repository
 - (b) Player: Movement
 - i. Mouse & Keyboard Controls
 - (c) UI: Basic UI
2. Oct 23, 2022
 - (a) UI: Room Reset
 - (b) Scene: Operating Room
 - i. Floor with Collision Box
 - ii. Operating Table
 - iii. Equipment Table(s)
 - (c) Player: Pick Up an Object
3. Oct 30, 2022
 - (a) Scene: Operating Room
 - i. Miscellaneous Equipment
 - (b) Player: Drop an Object
 - i. Collection Zone Collisions
 - ii. Collection Special Effect Particles for Success
 - iii. No Objects on the Floor
 - A. Negative Sound
 - B. Reset Room
4. Nov 6, 2022
 - (a) UI
 - i. Instructions Overlay
 - ii. Score (for each Success)
 - i. Collection Sound
 - A. Positive
 - B. Negative

5. Nov 13, 2022
 - (a) Procedural Operating Room
 - i. Randomized Object Positioning
 - ii. Randomized Instruction Set
6. Nov 20, 2022
 - (a) Integration Testing with Virtual Reality Hardware
 - (b) Virtual Real
 - (c) VR Integration Tests (Meta Quest 2 / Google Daydream)
 - (d) VR Controller Integrations
7. Nov 30, 2022
 - (a) Bug Fixes
 - (b) Final Project Report
 - (c) Final Project Deliverable
 - (d) Final Project Video

4.8 Deviations from original plan

Deviations from the original plan resulted due to the complexity of the task, the time frame, and a lack of 3D model assets. The main deviation is not implementing the interface guiding the player in the required tasks. While the chief purpose of the project is a proof of concept of integration of a Unity 3D game in a medical setting with a wearable VR technology, the player interface was deemed as a lower priority aspect of the project and thus delegated to next steps. Effort was re-allocated to providing visual feedback into the scene itself through particle systems when the user collected items, adding growing and shrinking orbs around the in-game collectible objects, and ensuring objects spawned in the scene oriented themselves in an intuitive way.

5 Division of Labour

In addition to the four main feature implementation areas described in the methodology section, the team will document the project and write final reports. The team's division of labor will fall into these five categories:

1. Operating Room Scene
 - (a) Nicholas Weber:
 - i. Responsible for the creating and managing the Unity project and repository.
 - ii. Responsible for code to procedurally generate the scene and its entities such as the gameobjects (for the player, tables, etc.).

- iii. Responsible for Entity Component Systems (ECS) and architecture of the app.

2. Player Controls

- (a) Nicholas Weber, Ali PiyarAli, & Josef Svoboda:
 - i. Responsible for writing and testing code scripts within the scenes related to player movements, actions to interact with the scene, and more.

3. User Interface

- (a) Nicholas Weber & Josef Svoboda:
 - i. Responsible for the user interface to instruct the player and provide non-action controls as necessary to play the game.

4. Hardware Integration

- (a) Nicholas Weber & Ali PiyarAli:
 - i. Responsible for hardware and software integrations using wearable technologies (probably Meta Quest 2 or Google Daydream).

5. Final Report(s)

- (a) Everyone:
 - i. The team will split responsibilities for documentation related to the project. The final project proposal, report, and video will be shared responsibilities of the group.

6 Results

(discuss the end results of your project and explain why they happened; include visuals and graphs here if applicable)

The outcome of the project is a game playable either as a desktop application or via a VR headset. The gameplay is demonstrated in the link in Project Repository and Demo.

Illustration the particular elements of the game:



Figure 1: Operating Room Scene 1.



Figure 2: Operating Room Scene 1.

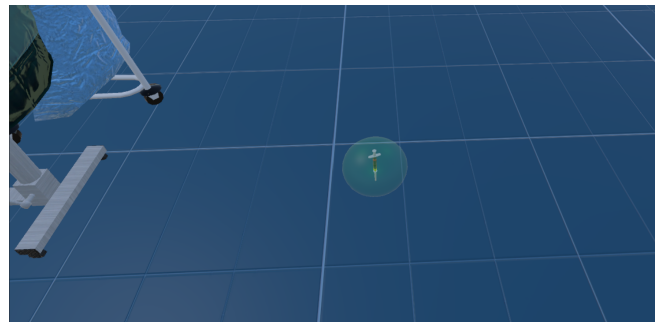


Figure 3: The game spawns objects which are to be picked up.



Figure 4: Object carried to a drop point.

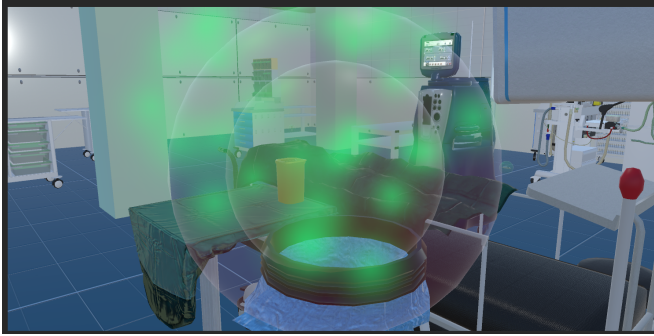


Figure 5: Object transferred to a drop point.

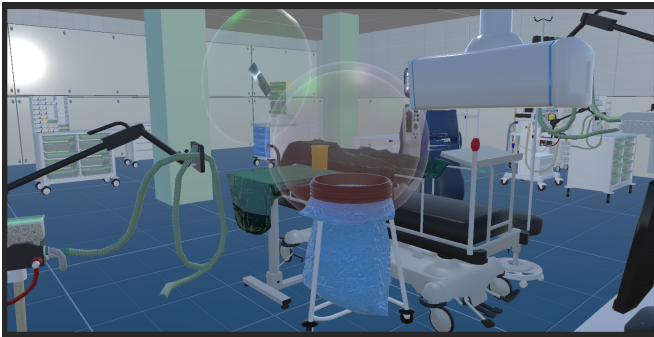


Figure 6: Object placed in a drop point, which is a bin.

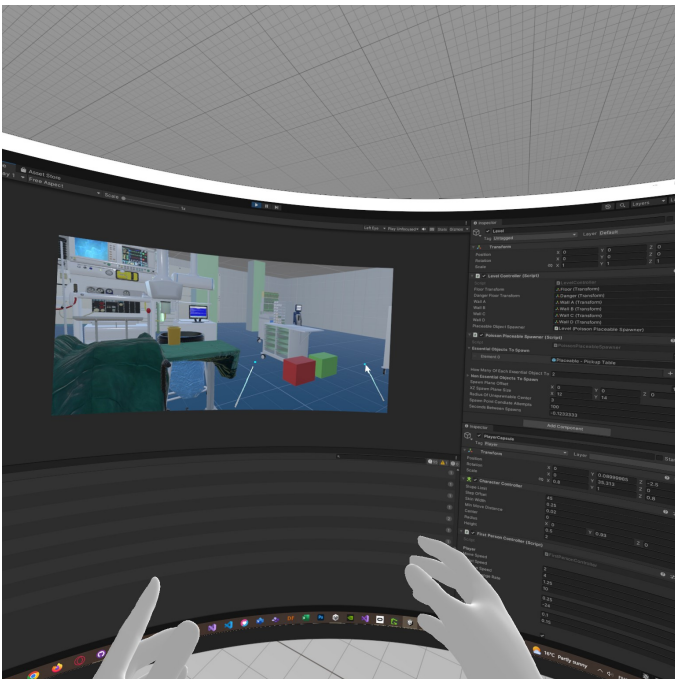


Figure 7: The game is integrated with a Meta Quest 2 VR headset; the image demonstrate the player interface experience

7 Next Steps

In order to develop the project into an actionable game, there are two necessary steps: not random objects but a specific object disappears when brought to the right space and second UI that gives specific instructions. The current project creates a scene of objects with a spherical drop-off position - any rigid body object dropped into gets deleted. The extension would require extending the behaviour of the drop-off point to make only specified objects disappear based on the state of the scene. Further extension would then be to create a user interface - such as prompts, sounds and effects - which would guide the user to drop off the objects in the right order.

Further steps would be to create scenes based on specific surgeries, possible extension of the scene to non-random placement of initial objects, as well as generation of animations, such as demonstrations of correct procedures for preparing an operating room for specific surgeries.

Examples of the surgical procedures include:

Appendectomy: This is a surgical procedure to remove the appendix, a small, tube-shaped organ that is part of the digestive system. To prepare the operating room for an appendectomy, the following steps can be taken:

- Clear the operating room of any unnecessary equipment or clutter, and make sure that all necessary equipment is in good working order and readily available.
- Set up the operating table and any additional equipment, such as a surgical lamp or suction device, according to the surgeon's preferences and the specific needs of the procedure.
- Cover the operating table with a clean, sterile surgical drape, and make sure that all instruments and supplies are sterile and ready for use.
- Gather any additional supplies that may be needed during the procedure, such as sutures, dressings, and antibiotics, and make sure they are within easy reach of the surgical team.

Coronary artery bypass graft (CABG): This is a surgical procedure to improve blood flow to the heart by creating a bypass around a blocked or narrowed coronary artery. To prepare the operating room for a CABG, the following steps can be taken:

- Clear the operating room of any unnecessary equipment or clutter, and make sure that all necessary equipment is in good working order and readily available.
- Set up the operating table and any additional equipment, such as a heart-lung machine or monitoring devices, according to the surgeon's preferences and the specific needs of the procedure.

- Cover the operating table with a clean, sterile surgical drape, and make sure that all instruments and supplies are sterile and ready for use.
- Gather any additional supplies that may be needed during the procedure, such as suture material, blood transfusion equipment, and drugs to support the heart and blood pressure, and make sure they are within easy reach of the surgical team.

The project can be then developed to encompass wider applicability – an example may be the teaching of first aid procedures, which is applicable to wider population beyond medical profession. Another way is to create educational games or quizzes that teach students about medical concepts and procedures. This can be done through interactive tutorials or quiz-style games.

The VR simulation for the purposes of medical education can be extended to anatomy teaching, for example by creating a scene inside a human body and assigning a role of specific cells to the player, interacting with its environment.

8 Conclusion

Our project involves the development of a Unity 3D game virtual reality operating room for use in medical education. By providing students with a simulated operating room environment, we hope to enhance their knowledge and comfort level in complex healthcare settings. Our project involves the creation of a VR experience that allows users to interact with a virtual operating room and complete tasks such as putting together a surgical tray for specific surgical procedures. We have integrated the project both as a desktop application and a VR experience with a Meta Quest 2 headset integration.

References

- Bridson, R. (2007). Fast Poisson Disk Sampling in Arbitrary Dimensions. Available at: <https://www.cs.ubc.ca/~rbridson/docs/bridson-siggraph07-poissondisk.pdf> (Accessed: 13 November 2022).
- Sharp Coder (2019). Unity FPS Controller. Available at: <https://sharpcoderblog.com/blog/unity-3d-fps-controller> (Accessed: 27 November 2022).
- Hsieh, M. & Lee, J. (2017) Preliminary Study of VR and AR Applications in Medical and Healthcare Education. Available at: <https://www.bvou.net/wp-content/uploads/2016/02/preliminary-study-of-vr-and-ar-applications-in-medical-and-healthcare-education...pdf>
- (Accessed: 26 October 2022).
- Margalit, Suresh, Marrache, Lentz, Lee, Tis, Varghese, Hayashi, Jain, Laporte. (2022). Evaluation of a Slipped Capital Femoral Epiphysis Virtual Reality Surgical Simulation for the Orthopaedic Trainee. Available at: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9042586/> (Accessed: 26 October 2022).
- Jaramillo, H. (2022). Easy Unity VR Tutorial for Beginners. Available at: <https://xrbootcamp.com/unity-vr-tutorial-for-beginners/#headline-59-465> (Accessed: 26 October 2022).
- Meta Quest Documentation. Get Started with Oculus in Unity. Available at: <https://developer.oculus.com/documentation/unity/unity-gs-overview/> (Accessed: 26 October 2022).
- Meta Quest Documentation. Grabbable. Available at: <https://developer.oculus.com/documentation/unity/unity-isdk-grabbable/> (Accessed: 6 December 2022).
- Catak, Masalci, Senyer. (2020). A Guideline Study For Designing Virtual Reality Games. Available at: https://www.researchgate.net/publication/348195955_A_Guideline_Study_For_Designing_Virtual_Reality_Games (Accessed: 26 October 2022).
- Rukas, M. (2022). PICKUP & DROP Physics Objects in Unity (Like PORTAL). Available at: <https://www.youtube.com/watch?v=6bFCQqabfzo> (Accessed: 25 November 2022).
- French, J. (2022). Raycasts in Unity, made easy. Available at: <https://gamedevbeginner.com/raycast-in-unity-made-easy/> (Accessed: 6 December 2022)
- Unity Documentation 2019.2 (2019). Input for Oculus. Available at: <https://docs.unity3d.com/2019.2/Documentation/Manual/OculusControllers.html> (Accessed: 6 December 2022)
- Unity Tutorial. (2022). Install the Unity Hub and Editor. Available at: <https://learn.unity.com/tutorial/install-the-unity-hub-and-editor#> (Accessed: 26 October 2022).